

Introdução a Persistência de Dados

Persistência de Dados em Aplicativos Mobile

O que é Persistência de Dados?

Persistência de dados é a **capacidade de um aplicativo armazenar informações de forma duradoura**, garantindo que esses dados permaneçam disponíveis mesmo após o app ser fechado ou o dispositivo reiniciado. Em aplicativos mobile, isso é essencial para oferecer uma experiência contínua ao usuário, sem perda de informações importantes.

Como Funciona a Persistência em Aplicativos Mobile?

Existem diferentes formas de manter os dados armazenados em dispositivos móveis. A escolha depende do tipo de informação, da frequência de uso e da necessidade de acesso online ou offline.

1. Armazenamento Interno

- Os dados são gravados diretamente no dispositivo.
- Ideal para **informações pessoais, preferências do usuário ou cache**.
- Tecnologias comuns:
 - **SharedPreferences (Android) / UserDefaults (iOS)**: para armazenar valores simples (ex: preferências).
 - **SQLite** ou **Moor (Flutter)**: bancos de dados locais para dados estruturados.
 - **Arquivos locais**: úteis para armazenar documentos, textos, imagens.

2. Armazenamento Externo / Remoto

- Os dados são armazenados fora do dispositivo, geralmente em servidores ou na nuvem.
- Ideal para **dados compartilhados entre dispositivos ou com acesso em tempo real**.
- Tecnologias comuns:

- **APIs REST/GraphQL** integradas com bancos de dados como Firebase, MySQL, PostgreSQL.
- **Serviços em nuvem**: como Firebase Firestore, AWS, Supabase, etc.

3. Banco de Dados Online (Realtime)

- Permite a **sincronização automática e em tempo real** dos dados.
 - Utilizado quando múltiplos usuários precisam visualizar e atualizar informações simultaneamente.
 - Exemplos: **Firebase Realtime Database**, **Firestore**, **Socket.IO** com Node.js.
-

Exemplos Práticos de Aplicação

Configurações do Aplicativo

- Exemplo: salvar o **idioma escolhido**, **tema (claro/escuro)** ou notificações.
- Solução: uso de SharedPreferences (ou shared_preferences no Flutter).

Lista de Tarefas (To-Do List)

- Exemplo: armazenar tarefas criadas pelo usuário.
- Solução: SQLite ou Hive (Flutter) para armazenar localmente.

Aplicativos de Mídia

- Exemplo: salvar **metadados de músicas, vídeos ou imagens**, permitindo carregamento rápido.
- Solução: banco local como SQLite ou leitura de diretórios com acesso a arquivos.

Redes Sociais

- Exemplo: armazenamento de **perfil, postagens e interações**.

- Solução: banco de dados remoto, acessado via **APIs REST**; uso de autenticação e cache local.

Considerações Finais

A persistência de dados é um pilar fundamental no desenvolvimento mobile, pois:

- Melhora a experiência do usuário.
- Permite funcionalidades offline.
- Facilita a personalização e continuidade no uso do app.

A escolha entre **armazenamento local ou remoto** deve levar em conta a sensibilidade dos dados, a necessidade de sincronização e os recursos do dispositivo.

Armazenamento com
shared_preferences no Flutter

shared_preferences no Flutter

O que é?

O `shared_preferences` é um pacote do Flutter utilizado para **armazenar dados simples de forma persistente** no dispositivo do usuário. Ele funciona como um **arquivo de chave-valor** (semelhante a um mini banco de dados) e é ideal para salvar **informações pequenas e rápidas**, como configurações, estados do app, preferências do usuário, etc.

Como Funciona?

O `shared_preferences` grava os dados diretamente no **armazenamento interno do dispositivo**. Esses dados:

- Permanecem salvos mesmo que o app seja fechado ou o celular reiniciado.
 - São armazenados em um formato leve e rápido de acessar.
 - Só podem ser acessados **pelo próprio aplicativo**.
-

Quando Usar?

Situação	Usar <code>shared_preferences</code> ?
Salvar nome de usuário	Sim
Salvar tema escuro/claro	Sim
Salvar uma lista de tarefas	Não Recomendado, mas pode (melhor usar <code>sqflite</code>)
Salvar uma imagem ou arquivos	Não (não recomendado)
Salvar dados sensíveis como senha	Não (use armazenamento seguro)

Exemplo Prático

https://docs.google.com/document/d/1S0aBlm4j_Qb94B5bL83BNwAcdlpCmpEtCSqNGqaUHHw/edit?tab=t.dkzb2zhc7wbq

Salvar e recuperar o nome do usuário

- ☐ Criar o aplicativo
- ☐ Instalar o shared_preferences nas dependencias
- ☐ criar o código

App com Tema Claro/Escuro Persistente

- ☐ Criar o aplicativo
- ☐ Instalar o shared_preferences nas dependencias
- ☐ criar o código

Lista de Tarefas com Shared Preferences

- ☐ Criar o aplicativo
- ☐ Instalar o shared_preferences nas dependencias
- ☐ criar o código

Benefícios

- Muito fácil de usar.
- Rápido para leitura e escrita.
- Sem necessidade de estrutura complexa de banco de dados.

Limitações

- Armazena apenas **tipos simples**: int, double, bool, String, List<String>.
 - Não é adequado para dados grandes ou complexos (listas com objetos, imagens, etc.).
 - Não é criptografado (evite armazenar dados sensíveis).
-

Exercício: Meu Perfil Persistente

Objetivo:

Criar um aplicativo onde o usuário pode preencher e salvar informações pessoais simples (nome, idade e cor favorita). Esses dados devem ser **armazenados usando shared_preferences** e carregados automaticamente ao abrir o aplicativo.

O app deve:

- Ter campos para digitar:
 - Nome
 - Idade
 - Cor favorita (escolhida por um botão ou dropdown)
- Ter um botão "Salvar Dados"
- Exibir os dados salvos abaixo dos campos (após o carregamento)
- Armazenar e carregar os dados usando shared_preferences
- Personalize o fundo do app de acordo com a cor favorita escolhida (usando o valor salvo).

Armazenamento com SqfLite em Flutter

Plano de Ensino:

Persistência de Dados com sqflite (SQLite no Flutter)

Objetivo Geral

Ao final deste módulo, o aluno será capaz de compreender o funcionamento de bancos de dados relacionais no contexto mobile, utilizar o plugin `sqflite` para integrar um banco SQLite a uma aplicação Flutter, definir modelos de dados e implementar as operações básicas de **CRUD** (Criar, Ler, Atualizar, Deletar).

1. Introdução a Bancos de Dados Relacionais no Mobile

O SQLite como Ferramenta de Armazenamento

O SQLite é um sistema de gerenciamento de banco de dados relacional (SGBD) embutido. Ele é o padrão da indústria para aplicativos móveis devido à sua leveza, simplicidade e capacidade de operar sem a necessidade de um servidor dedicado.

Por que usar o SQLite em aplicativos móveis?

- **Embutido:** Integrado diretamente ao aplicativo, dispensando instalações separadas ou configurações de rede.
- **Leve:** Consome o mínimo de recursos de hardware (memória e processamento), ideal para dispositivos móveis.

- **Transacional (ACID):** Garante a integridade dos dados mesmo em cenários de falha ou fechamento abrupto do app.
- **Compatível:** Utiliza a sintaxe padrão do SQL, facilitando o reaproveitamento de conhecimento prévio.

Possibilidades de Uso do SQLite

- **Armazenamento Estruturado:** Ideal para informações de perfil, configurações do app e históricos.
 - **Arquitetura Offline-First:** Permite que aplicativos (como listas de tarefas e catálogos) funcionem perfeitamente sem internet.
 - **Cache de Dados:** Armazena localmente dados vindos de APIs externas para acelerar o carregamento da interface.
 - **Gerenciamento de Relacionamentos:** Suporta chaves primárias e estrangeiras para modelar dados complexos.
-

2. Configurando o `sqflite` no Projeto Flutter

Para habilitar o suporte ao SQLite no Flutter, utilizamos o pacote principal `sqflite`. Como os dados precisam ser persistidos no armazenamento físico do dispositivo, utilizamos os pacotes auxiliares `path` e `path_provider` para gerenciar os diretórios do sistema de arquivos.

Adicione as dependências executando o comando no terminal:

```
None  
flutter pub add sqflite path path_provider
```

Entendendo os Pacotes Auxiliares

- `path`: Biblioteca Dart que fornece utilitários robustos para manipulação e concatenação de caminhos de arquivos e diretórios de forma multiplataforma.
- `path_provider`: Permite acessar de forma segura e padronizada os diretórios nativos do Android e iOS.

****Principais Diretórios do `path_provider`****

Método	Descrição / Caso de Uso
<code>getApplicationDocumentsDirectory()</code>	Recomendado para o SQLite. Retorna o diretório exclusivo da aplicação onde os dados gerados pelo usuário não devem ser apagados pelo sistema.
<code>getApplicationSupportDirectory()</code>	Diretório para arquivos de suporte que o app precisa para funcionar, mas que não devem ser expostos ao usuário.
<code>getTemporaryDirectory()</code>	Diretório de cache temporal. O sistema operacional pode apagar esses arquivos se precisar de espaço livre.
<code>getExternalStorageDirectory()</code>	Acesso ao armazenamento externo (como cartões SD). Disponível primariamente no Android.

3. Modelando Dados em Dart

O `sqflite` não armazena objetos Dart diretamente; ele interage com o banco de dados utilizando **Mapas** (`Map<String, dynamic>`) e comandos SQL nativos. A modelagem de dados cria a ponte (Mapeamento Objeto-Relacional) entre suas classes Dart e as tabelas SQL.

Passo 1: Definindo o Schema do Banco de Dados

Consiste em traduzir as propriedades da sua classe Dart para colunas e tipos equivalentes no SQLite.

Equivalência de Tipos Comuns:

- `INTEGER`: Para números inteiros (IDs, contadores, booleanos).
- `TEXT`: Para cadeias de caracteres (Strings, datas formatadas).
- `REAL`: Para números de ponto flutuante (`double`).

Modelo em Dart:

None

```
class Nota {  
  final int? id;  
  final String titulo;  
  final String conteudo;  
  
  Nota({this.id, required this.titulo, required this.conteudo});  
}
```

Schema Correspondente em SQL:

None

```
CREATE TABLE notas (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  titulo TEXT NOT NULL,  
  conteudo TEXT  
);
```

Passo 2: Definindo o Mapeamento Objeto-Relacional

Para automatizar a conversão, adicionamos os métodos `toMap()` e `fromMap()` dentro da própria classe de modelo.

None

```
class Nota {  
    final int? id;  
    final String titulo;  
    final String conteudo;  
  
    Nota({this.id, required this.titulo, required this.conteudo});  
  
    // Converte o Objeto Dart -> Map (Preparação para  
    Inserção/Atualização)  
    Map<String, dynamic> toMap() {  
        return {  
            'id': id, // 0 ID pode ser nulo se o banco for gerá-lo  
            'titulo': titulo,  
            'conteudo': conteudo,  
        };  
    }  
  
    // Converte o Map -> Objeto Dart (Instanciação após Leitura do  
    Banco)  
    factory Nota.fromMap(Map<String, dynamic> map) {  
        return Nota(  
            id: map['id'] as int?,  
            titulo: map['titulo'] as String,  
            conteudo: map['conteudo'] as String,  
        );  
    }  
}
```

```
}  
  
}
```

Tratando Tipos de Dados Específicos

Como o SQLite não possui tipos nativos para `bool` ou `DateTime`, precisamos convertê-los antes de salvar e ao recuperar.

- **Booleanos** (`bool`): Salvos como `INTEGER` (`1` para verdadeiro, `0` para falso).
- **Datas** (`DateTime`): Salvas como `TEXT` utilizando o formato ISO 8601 (`YYYY-MM-DD HH:MM:SS.SSS`).

Exemplo Prático: Classe Tarefa

None

```
class Tarefa {  
    final int? id;  
    final String descricao;  
    final bool concluida;  
    final DateTime dataCriacao;  
  
    Tarefa({  
        this.id,  
        required this.descricao,  
        required this.concluida,  
        required this.dataCriacao,  
    });  
  
    Map<String, dynamic> toMap() {
```

```
return {
    'id': id,
    'descricao': descricao,
    'concluida': concluida ? 1 : 0, // Conversão bool -> int
    'data_criacao': dataCriacao.toIso8601String(), // Conversão
    DateTime -> String
};
}

factory Tarefa.fromMap(Map<String, dynamic> map) {
    return Tarefa(
        id: map['id'] as int?,
        descricao: map['descricao'] as String,
        concluida: map['concluida'] == 1, // Conversão int -> bool
        dataCriacao: DateTime.parse(map['data_criacao'] as String),
        // Conversão String -> DateTime
    );
}
}
```

4. Estrutura do Banco de Dados: A Classe DatabaseHelper

Para gerenciar a conexão com o banco de dados, centralizamos a lógica em uma classe dedicada aplicando o padrão de projeto **Singleton**. Isso garante uma única instância de conexão aberta durante todo o ciclo de vida do app.

Vantagens da Abordagem:

1. **Instância Única (Singleton):** Evita conflitos de leitura/escrita concorrente e economiza memória.
 2. **Inicialização Assíncrona (Lazy Initialization):** O banco só é aberto no momento em que é requisitado pela primeira vez.
 3. **Propagação de Erros:** Exceções são repassadas para a camada de Interface de Usuário (UI) decidir como alertar o usuário.
-

5. Gerenciando o Banco de Dados com sqflite

Fluxo de Inicialização

- `openDatabase()`: Localiza o arquivo `.db` no diretório seguro do dispositivo.
- **Callback `onCreate`**: Executado apenas na primeira vez que o aplicativo é aberto para estruturar as tabelas.
- **Callback `onUpgrade`**: Acionado quando a versão do banco (`version`) é incrementada, permitindo a execução de scripts de migração (ex: adicionar colunas).

Implementação das Operações CRUD (Assíncronas)

- **Criar (`Insert`):** Recebe o `Map` do objeto através do método `.insert()`. Retorna o ID (inteiro) da linha recém-criada.
- **Ler (`Query`):** Utiliza `.query()`. Retorna uma `List<Map<String, dynamic>>`. Requer o mapeamento de volta para objetos Dart usando o construtor `fromMap`. Suporta cláusulas estruturadas como `WHERE`, `ORDER BY`, `LIMIT` e `OFFSET`.

- **Atualizar (Update):** Altera registros existentes via `.update()`. **Atenção:** Sempre utilize cláusulas `where` e `whereArgs` parametrizadas para evitar falhas de segurança (SQL Injection). Retorna o número de linhas afetadas.
 - **Deletar (Delete):** Remove dados com `.delete()`. Também exige parametrização com `where` e `whereArgs`. Retorna a quantidade de linhas removidas.
-

6. Integração com a Interface do Usuário (UI)

- **Gerenciamento de Estado:** Recomenda-se integrar a classe Helper com soluções de estado (como *Provider*, *Riverpod*, ou *Bloc*) para evitar chamadas diretas da camada visual.
 - **Exibição Assíncrona:** Utilização de `FutureBuilder` para lidar com o estado de carregamento inicial dos dados ou atualização manual via `setState` após operações de escrita.
 - **Componentização:** Renderização eficiente de coleções através do widget `ListView.builder`.
-

7. Considerações Avançadas (Opcional / Próximos Passos)

- Execução de Queries em SQL puro através de métodos `rawInsert()`, `rawQuery()`, etc.
 - Uso de **Transações** (`database.transaction()`) para garantir atomicidade em operações em lote (tudo ou nada).
 - Introdução a relacionamentos complexos utilizando cláusulas `JOIN`.
-

8. Exercícios Práticos de Aplicação

Os exercícios abaixo visam consolidar o ciclo do CRUD utilizando a arquitetura encapsulada na classe Helper.

Exercício 1: Lista de Itens Simples (CRUD Básico - C e R)

- **Objetivo:** Implementar uma tela que exibe uma lista de itens salvos no banco de dados e possibilite a adição de novos registros.
- **Descrição:**
 1. Crie o modelo `Item` com `id` (int), `nome` (String) e `quantidade` (int).
 2. Crie a classe `ItemDatabaseHelper` com o método `_onCreate` configurando a tabela `itens`.
 3. Implemente os métodos `insertItem(Item item)` e `getItems()` (retornando `Future<List<Item>>`).
 4. Na UI (`StatefulWidget`), carregue a lista no método `initState` e exiba os dados em um `ListView.builder`.
 5. Adicione um `FloatingActionButton` que abra um diálogo simples para salvar um novo item e force a atualização da lista.

Exercício 2: Adicionando Funcionalidade de Exclusão (D)

- **Objetivo:** Estender o Exercício 1 adicionando a capacidade de remover itens do banco.
- **Descrição:**
 1. No `ItemDatabaseHelper`, crie o método `deleteItem(int id)`.
 2. Na interface gráfica, insira um botão de exclusão (ex: ícone de lixeira dentro do `ListTile` ou envolvendo-o com o widget `Dismissible` para exclusão via gesto de arrastar).
 3. Ao acionar o comando, invoque o método de remoção e atualize o estado da tela para refletir a mudança.

Exercício 3: Adicionando Funcionalidade de Atualização (U)

- **Objetivo:** Estender o Exercício 2 permitindo a edição de itens existentes.

- **Descrição:**

1. No `ItemDatabaseHelper`, implemente o método `updateItem(Item item)`.
2. Na UI, configure o clique no item (`onTap`) para abrir uma janela de edição preenchida com os dados atuais do item selecionado.
3. Após confirmar as alterações, execute a persistência e atualize a listagem.

Exercício 4 (Desafio): Filtro e Busca Dinâmica

- **Objetivo:** Implementar uma barra de pesquisa para filtrar elementos diretamente no banco de dados.

- **Descrição:**

1. No `ItemDatabaseHelper`, crie o método `searchItems(String query)`.
2. A busca deve mapear o banco utilizando cláusulas estruturadas com o operador `LIKE` (ex: `WHERE nome LIKE ?`, passando no argumento: `['%$query%']`).
3. Na `AppBar` da interface do usuário, adicione um campo de texto (`TextField`). Conforme o texto for alterado pelo usuário, dispare a consulta de filtro e atualize a tela em tempo real. Se o campo for esvaziado, exiba todos os itens novamente.

SA Formativa - PETSHOP

Situação de Aprendizagem Formativa: Gerenciamento de Consultas de PetShop com Flutter e SQLite

Nesta situação de aprendizagem, vamos desenvolver um **aplicativo prático e funcional** para um PetShop. Nosso objetivo é criar um sistema simples de gerenciamento de consultas, onde poderemos **cadastrar pets**, visualizar seus detalhes e **agendar consultas** para cada um deles. Para a persistência dos dados (ou seja, para que as informações não se percam ao fechar o app), utilizaremos o **SQLite**, um banco de dados leve e eficiente, integrado ao Flutter através da biblioteca `sqflite`.

Estrutura do Projeto: Organizando o Código

A organização é fundamental em qualquer projeto de software. Seguiremos uma estrutura clara para manter nosso código limpo e de fácil manutenção:

- **lib/**: Este é o diretório principal do seu código-fonte Dart.
 - **main.dart**: O ponto de entrada do seu aplicativo, onde tudo começa.
 - **controllers/**: Aqui residirão as **lógicas de negócio** e a comunicação entre as telas e o banco de dados. Teremos um controle para as operações de **Pet** e outro para as de **Consulta**.
 - **models/**: Neste diretório, definiremos as **estruturas dos dados** que serão manipulados, como as classes `Pet` e `Consulta`. Elas representam como os dados são organizados e armazenados.
 - **database/**: Contém o "**cérebro**" da interação com o banco de dados, o `database_helper.dart`. Ele será responsável por abrir, criar as tabelas e realizar todas as operações de leitura, escrita, atualização e exclusão.
 - **screens/**: Onde as **telas (interfaces de usuário)** do aplicativo serão construídas. Cada arquivo aqui representará uma parte visual e interativa do nosso app.
 - `home_screen.dart`: A tela principal, que exibirá uma **lista de todos os pets cadastrados**.
 - `pet_detail_screen.dart`: Ao selecionar um pet, esta tela mostrará seus **detalhes específicos** e todas as **consultas agendadas** para ele.
 - `add_pet_screen.dart`: Um **formulário intuitivo** para o usuário **cadastrar um novo pet**.
 - `add_consulta_screen.dart`: Outro **formulário**, dedicado ao **agendamento de novas consultas** para um pet específico.
- **pubspec.yaml**: Um arquivo crucial que gerencia as **dependências** do seu projeto Flutter, onde declaramos as bibliotecas externas que utilizaremos.

Detalhamento das Etapas:

Passo 1: Configurar Dependências

Antes de começar a codificar as funcionalidades, precisamos informar ao Flutter quais "pacotes" externos usaremos. No arquivo `pubspec.yaml`, adicionaremos as seguintes bibliotecas:

- **sqflite**: A biblioteca essencial para **interagir com o banco de dados SQLite**.
- **path**: Ajuda a **gerenciar os caminhos de arquivos** no sistema, algo importante para localizar o banco de dados.
- **path_provider**: Fornece os **caminhos específicos do sistema operacional** onde podemos armazenar dados, como o nosso banco SQLite.
- **intl**: Uma biblioteca para **formatar datas e horas** de forma localizada, garantindo que as informações de tempo das consultas sejam exibidas de maneira amigável.

Passo 2: Modelar os Dados

Aqui, definiremos como nossos dados serão estruturados. Criaremos classes Dart que representam as "tabelas" do nosso banco de dados:

- **Pet**: Esta classe representará um pet individual, com atributos como nome, raça, nome do dono e telefone do dono.
- **Consulta**: Esta classe representará uma consulta agendada, contendo informações como o tipo de serviço, data e hora, observações, e, crucialmente, uma referência ao **ID do pet** ao qual ela pertence (estabelecendo um relacionamento entre Pet e Consulta).

Para que essas classes possam ser armazenadas e recuperadas do banco de dados, elas incluirão métodos especiais para converter seus objetos em um formato de `Map<String, dynamic>` (que o banco de dados entende) e vice-versa.

Passo 3: Criar o Database Helper

Este é o componente que faz a "ponte" entre o seu aplicativo e o banco de dados SQLite. O `DatabaseHelper` será uma classe que:

- **Inicializará o banco de dados**: Abrindo um banco existente ou criando um novo se for a primeira vez que o aplicativo é executado.
- **Definirá as tabelas**: Irá criar as tabelas `pets` e `consultas` no banco, com as colunas e tipos de dados corretos.
- **Implementará as operações CRUD**: Isso significa que ele conterá os métodos para:
 - **Create** (inserir novos pets e consultas)

- **Read** (listar todos os pets, buscar um pet por ID, listar consultas de um pet)
- **Update** (atualizar informações de pets ou consultas)
- **Delete** (remover pets ou consultas)

Passo 4: Criar os Controllers

Os controllers atuam como uma camada intermediária entre a interface do usuário (telas) e a lógica de persistência (Database Helper). Eles são responsáveis por:

- **PetController**: Gerenciar as operações relacionadas aos pets, como adicionar um novo pet, buscar pets e outras interações diretas com a entidade Pet.
- **ConsultaController**: Gerenciar as operações relacionadas às consultas, como agendar uma nova consulta, listar consultas de um pet específico e outras interações com a entidade Consulta.

Essa separação de responsabilidades torna o código mais modular e organizado.

Passo 5: Criar a UI (Interface de Usuário)

Por fim, construiremos as telas do nosso aplicativo. Cada tela será responsável por uma parte específica da interação com o usuário, desde a exibição de listas até a entrada de dados em formulários. As telas utilizarão os controllers para realizar as operações de negócio e interagir com o banco de dados, exibindo os resultados para o usuário.

Este projeto oferece uma excelente oportunidade para entender a **arquitetura de um aplicativo Flutter** com persistência de dados local, a importância da **separação de responsabilidades** (models, controllers, database helper, UI) e a **interação com um banco de dados relacional** como o SQLite.

SA Somativa - SQLite/Modelagem

Situação de Aprendizagem Somativa:

Desenvolvimento de Aplicações Mobile com Persistência de Dados

Apresentação da Atividade

Nesta atividade, você terá a oportunidade de aplicar seus conhecimentos em desenvolvimento mobile com Flutter para criar soluções práticas e funcionais. O foco principal será a implementação de sistemas com persistência de dados local, utilizando o banco de dados SQLite para armazenar e gerenciar informações de forma eficiente.

Sua proposta será sorteada entre as 17 briefings de projetos apresentados, cada um projetado para simular cenários reais de aplicação. Cada projeto segue um fluxo comum e essencial em muitos softwares: um registro inicial de um item principal, a visualização detalhada desse registro e a capacidade de adicionar elementos relacionados a ele. Essa abordagem visa consolidar seu entendimento sobre modelagem de dados, interações com banco de dados e a construção de interfaces de usuário dinâmicas.

Objetivos da Atividade

Ao final desta situação de aprendizagem, espera-se que você seja capaz de:

- Aplicar conceitos de desenvolvimento Flutter: Utilizar os widgets e a arquitetura do Flutter para construir interfaces de usuário responsivas e interativas.
- Modelar dados: Criar classes e estruturas de dados apropriadas para representar as entidades de um sistema.
- Implementar persistência de dados local: Configurar e interagir com o banco de dados SQLite usando a biblioteca sqflite para armazenar, recuperar, atualizar e excluir informações.

- Gerenciar o estado da aplicação: Desenvolver lógicas para atualizar a interface do usuário dinamicamente com base nas interações do usuário e nos dados do banco.
 - Organizar o código: Estruturar o projeto de forma modular, separando responsabilidades (modelos, controllers, helpers de banco de dados e telas) para facilitar a manutenção e escalabilidade.
 - Resolver problemas de lógica de negócio: Implementar regras e validações que garantam a integridade e o comportamento esperado do aplicativo.
 - Desenvolver habilidades de depuração: Identificar e corrigir erros no código.
-

Competências Abordadas

Esta atividade aborda as seguintes competências essenciais para o desenvolvimento de software:

- Programação Orientada a Objetos (POO): Aplicação de princípios como encapsulamento, herança e polimorfismo na modelagem de classes e estruturas.
 - Estrutura de Dados e Algoritmos: Compreensão e uso adequado de listas, mapas e outras estruturas para organizar informações.
 - Persistência de Dados: Habilidade em integrar e manipular bancos de dados relacionais (SQLite) em aplicações mobile.
 - Desenvolvimento Widgets (UI/UX): Construção de interfaces de usuário que sejam intuitivas, acessíveis e eficientes para o usuário final.
 - Arquitetura de Software: Capacidade de projetar e organizar componentes de software em camadas lógicas para modularidade e reusabilidade.
 - Resolução de Problemas: Abordagem sistemática para identificar, analisar e resolver desafios técnicos.
 - Trabalho Individual/Autonomia: Gerenciamento do próprio processo de desenvolvimento e busca por soluções de forma independente.
-

CrITÉrios de Avaliação

Seu projeto será avaliado com base nos seguintes critérios:

1. **Funcionalidade:**

- Todas as funcionalidades essenciais do projeto escolhido estão implementadas e operacionais (cadastro, visualização, adição de elementos relacionados).
- A persistência de dados está funcionando corretamente (os dados são salvos e carregados ao reiniciar o aplicativo).
- Validações de formulário e tratamento de erros básicos estão presentes e são eficazes.

2. **Qualidade do Código:**

- Organização: A estrutura do projeto segue as convenções recomendadas (separação de models, controllers, database, screens).
- Clareza e Legibilidade: O código é fácil de ler e entender, com nomes de variáveis e funções significativos.
- Boas Práticas: Uso adequado de const, final, tipagem forte e outros recursos da linguagem Dart/Flutter.
- Tratamento de Exceções: Uso de try-catch quando apropriado para lidar com possíveis erros.

3. **Interface do Usuário (UI) e Experiência do Usuário (UX):**

- Layout: A interface é visualmente organizada e bem espaçada.
- Navegação: A transição entre as telas é clara e intuitiva.
- Feedback ao Usuário: Mensagens de sucesso ou erro são exibidas de forma clara.

4. **Entrega e Documentação:**

- O projeto foi entregue dentro do prazo estabelecido.
 - O código está disponível em um repositório (ex: GitHub) de forma organizada.
 - (Opcional) Breve README.md explicando como rodar o projeto e as principais funcionalidades.
-

Projetos:

1. Sistema de Gerenciamento de Biblioteca (e-books e físicos) Raissa - Vitor Gabriel - Leonardo - Julya - Esther

Fluxo:

1. Registro Inicial: Cadastro de Livros (Título, autor, ISBN, ano, editora, gênero, tipo (físico/e-book), quantidade disponível, capa).
 2. Visualização do Registro: Detalhes do Livro selecionado, exibindo todas as suas informações e, para livros físicos, o histórico de empréstimos/devoluções.
 3. Adição de Elementos ao Registro: Para livros físicos, Registrar Empréstimo (data de empréstimo, previsão de devolução, nome do locatário) ou Registrar Devolução.
-

2. Aplicativo de Controle Financeiro Pessoal (Pedro) - thauany - Miguel - Paraluppi - Mito

Fluxo:

1. Registro Inicial: Cadastro de Categorias (ex: Alimentação, Transporte, Lazer, Salário). O usuário cria suas próprias categorias de despesas/receitas.
 2. Visualização do Registro: Detalhes da Categoria, mostrando o total de despesas e receitas associadas a ela em um período.
 3. Adição de Elementos ao Registro: Para uma categoria específica, Adicionar Transação (valor, descrição, data, tipo: receita/despesa).
-

3. Gerenciador de Tarefas (To-Do List Avançado) Paula - Maria - Enzo Donizette - Gulitti

Fluxo:

1. Registro Inicial: Criação de Listas/Projetos (ex: Trabalho, Pessoal, Compras, Reforma da Casa).
 2. Visualização do Registro: Detalhes da Lista/Projeto, exibindo todas as tarefas pertencentes a ela.
 3. Adição de Elementos ao Registro: Para uma lista/projeto selecionado, Adicionar Nova Tarefa (título, descrição, data de vencimento, prioridade, status).
-

4. Diário de Viagens - Manfré - Kaua Rodrigues - Guilherme - Iasmin

Fluxo:

1. Registro Inicial: Planejar Nova Viagem (Título da viagem, destino, datas de início/fim, descrição).
 2. Visualização do Registro: Detalhes da Viagem, mostrando as informações gerais e uma linha do tempo das entradas diárias.
 3. Adição de Elementos ao Registro: Para a viagem selecionada, Adicionar Entrada Diária (data, texto do diário, opção de adicionar foto).
-

6. App de Treinos Físicos Personalizados - Vitor - PedroHP - Beatriz - Isabela Romão

Fluxo:

1. Registro Inicial: Criar Nova Rotina de Treino (Nome da rotina, objetivo).
 2. Visualização do Registro: Detalhes da Rotina, listando os exercícios que a compõem.
 3. Adição de Elementos ao Registro: Para a rotina selecionada, Adicionar Exercício à Rotina (nome do exercício, séries, repetições, carga, tipo).
-

7. Controle de Estoque Simples para Pequeno Negócio - Suemi - Matheus - Adrielly - Giovanna Generozo

Fluxo:

1. Registro Inicial: Cadastro de Produto (Nome, descrição, preço de custo, preço de venda, quantidade inicial em estoque, código).
 2. Visualização do Registro: Detalhes do Produto, exibindo informações gerais e o histórico de entrada/saída.
 3. Adição de Elementos ao Registro: Para o produto selecionado, Registrar Movimentação de Estoque (tipo: entrada/saída, quantidade, data, observação).
-

8. Sistema de Agendamento - Consultório Médico - Luiza Félix - PedroHG - Derick - Isabela Lopes

Fluxo:

1. Registro Inicial: Cadastro de Paciente (Nome completo, CPF, data de nascimento, telefone, e-mail).
2. Visualização do Registro: Perfil do Paciente, exibindo seus dados pessoais e um histórico das consultas agendadas e realizadas.
3. Adição de Elementos ao Registro: Para o paciente selecionado, Agendar Nova Consulta (data, hora, especialidade/tipo de consulta, observações).

9. Sistema de Agendamento - Consultório Odontológico - bull - Sabino - Isabela - Espadoni

Fluxo:

1. Registro Inicial: Cadastro de Paciente (Nome completo, CPF, data de nascimento, telefone, e-mail, histórico médico relevante).
2. Visualização do Registro: Prontuário do Paciente, mostrando seus dados e o histórico de atendimentos e procedimentos realizados.
3. Adição de Elementos ao Registro: Para o paciente selecionado, Registrar Novo Atendimento (data, hora, descrição do procedimento, dentes envolvidos, observações, valor cobrado).

10. Gerenciador de Coleções (Ex: Selos, Moedas, HQs) - Mirella - Vitor Rossi - Anick - Giovana Furlanete

Fluxo:

1. Registro Inicial: Criação de Coleção (Nome da coleção, descrição, tipo de item (ex: Selos, Moedas, HQs)).
2. Visualização do Registro: Detalhes da Coleção, exibindo todos os itens catalogados nela.
3. Adição de Elementos ao Registro: Para a coleção selecionada, Adicionar Novo Item (nome do item, descrição, data de aquisição, valor, condição, foto, detalhes específicos da categoria - ex: país da moeda).

11. Calculadora de IMC e Histórico de Peso - Crippa - Nathaly - Eduardo - Laiza

Fluxo:

1. Registro Inicial: Cadastro de Perfil (Nome, altura, sexo, data de nascimento).
2. Visualização do Registro: Detalhes do Perfil, exibindo altura, IMC atual e um gráfico/lista do histórico de peso.
3. Adição de Elementos ao Registro: Para o perfil selecionado, Registrar Novo Peso (valor do peso, data da medição).

12. Gerenciador de Contatos Telefônicos - Tiago - Pedro Caversan - João Cardoso-Julia(castelo)

Fluxo:

1. Registro Inicial: Cadastro de Contato (Nome, telefone principal, e-mail).
 2. Visualização do Registro: Detalhes do Contato, mostrando todas as suas informações.
 3. Adição de Elementos ao Registro: Para o contato selecionado, Adicionar Campo Adicional (ex: outro telefone, endereço, aniversário, observações).
-

13. Aplicativo de Recebimento de Pedidos (Simples) - Pietro - Rafael - Clara - Gabriel Dias

Fluxo:

1. Registro Inicial: Cadastro de Produto no Menu (Nome, descrição, preço, categoria).
 2. Visualização do Registro: Menu de Produtos, exibindo os produtos disponíveis para pedido.
 3. Adição de Elementos ao Registro: Para cada produto, Adicionar ao Carrinho (selecionar quantidade e adicionar ao pedido em andamento). O "registro" final aqui seria o pedido.
-

14. Controle de Manutenção de Veículos - Rennan - Lucas Pains - Julia Oliveira - Daniel

Fluxo:

1. Registro Inicial: Cadastro de Veículo (Marca, modelo, ano, placa, quilometragem inicial).
 2. Visualização do Registro: Detalhes do Veículo, exibindo seus dados e o histórico de manutenções realizadas.
 3. Adição de Elementos ao Registro: Para o veículo selecionado, Registrar Nova Manutenção (tipo de serviço, data, quilometragem no momento da manutenção, custo, observações).
-

15. Registro de Plantas (Jardim Virtual) - Marco - Luisa - Endrew - Anthony

Fluxo:

1. Registro Inicial: Cadastro de Planta (Nome da planta, espécie, data de aquisição, local, foto).
 2. Visualização do Registro: Ficha da Planta, exibindo todas as suas informações e histórico de cuidados.
 3. Adição de Elementos ao Registro: Para a planta selecionada, Registrar Cuidado (tipo de cuidado: rega, adubação, poda; data, observações).
-

16. Controle de Entrada de Visitante - Maria - Lucas Dibber - Gleice

Fluxo:

1. Registro Inicial: Cadastro do Visitante (Nome, Documento, Idade, Endereço).
2. Visualização do Registro: Ficha do Visitante, exibindo todas as suas informações e histórico de visitas.
3. Adição de Elementos ao Registro: Para o visitante selecionado, Registrar Entrada (data da entrada, Saida , motivo da visita).

Entrega -

SESI A -

<https://classroom.google.com/c/NzQ5Nzg4MjM4MTc2/a/NzAwMDM4ODA5Mjcx/details>

SESI B -

<https://classroom.google.com/c/NzQ5Nzg4MjM4MTc2/a/NzAwMDU4MjY2OTMz/details>